45. Design Ad Bidding System

Key Terms:

Term	Full Name	What it does
RTB	Real-Time Bidding	Auction for ad impressions in milliseconds
DSP	Demand-Side Platform	Advertiser's bidding system
SSP	Supply-Side Platform	Publisher's ad inventory system
pCTR	Predicted Click-Through Rate	ML model predicting click probability
pCVR	Predicted Conversion Rate	ML model predicting conversion probability
CPM	Cost Per Mille	Price per 1000 impressions
CPC	Cost Per Click	Price per click
Bid Shading	Bid reduction	Bidding below value in first-price auctions
Pacing	Budget spread	Spending budget evenly throughout day
Frequency Cap	Impression limit	Max times user sees same ad

Problem Statement

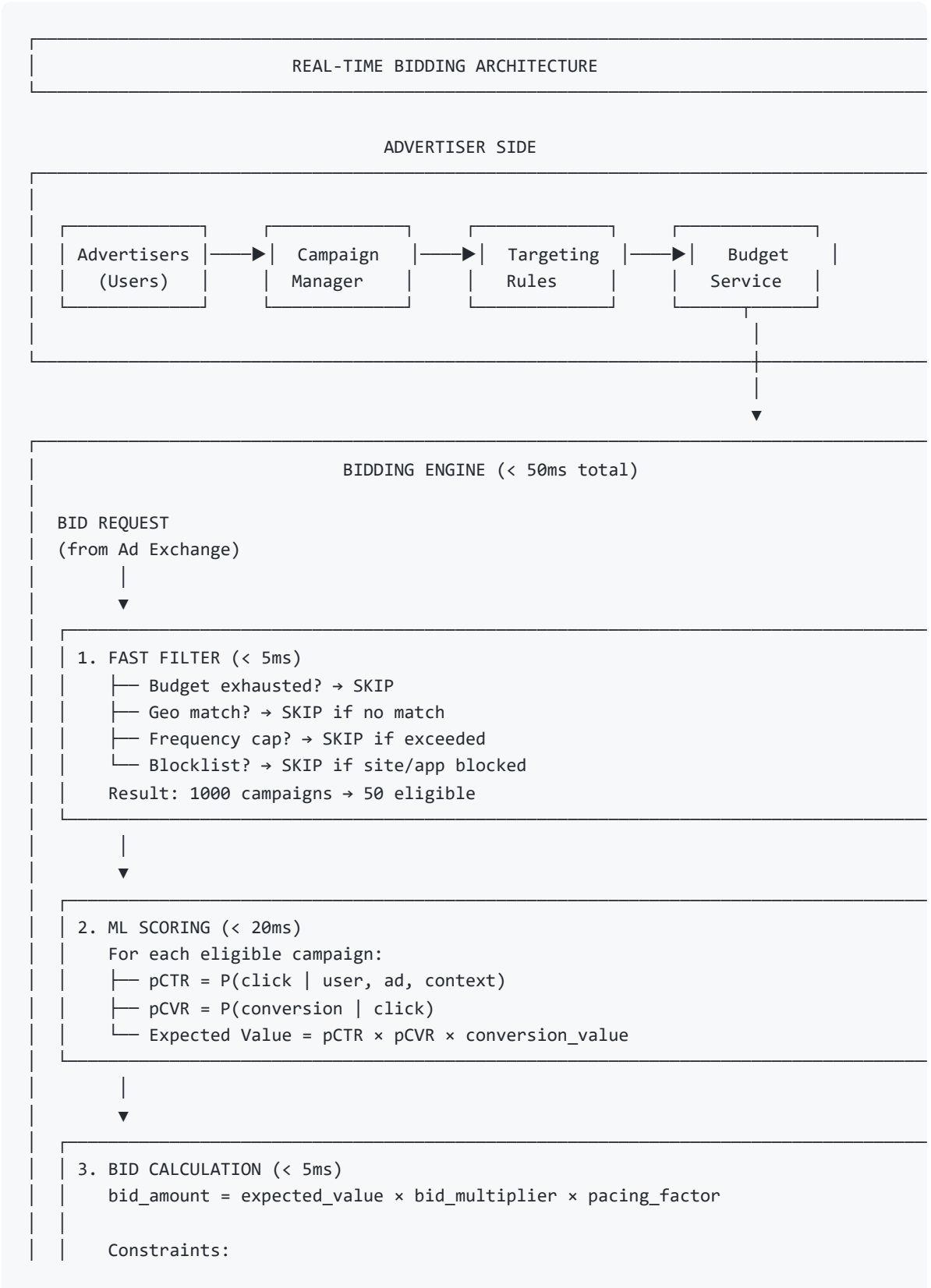
Design a real-time bidding (RTB) system that auctions ad impressions in milliseconds, like Google Ads, Microsoft Ads, or Meta Ads.

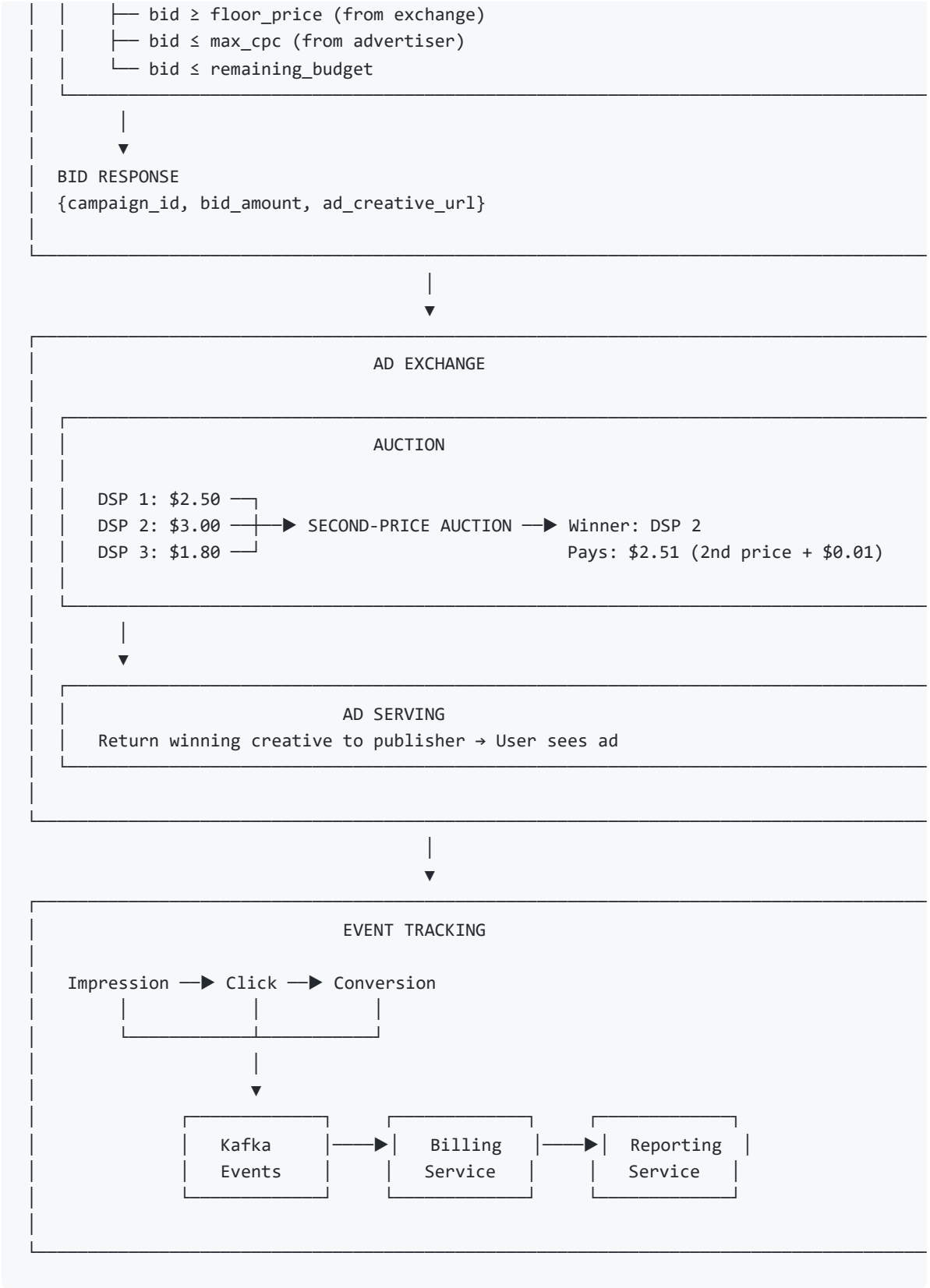
Clarifying Questions to Ask

1. **Latency**: What's the bid response deadline? (50-100ms typical)
2. **Scale**: How many bid requests per second? (1M+ QPS?)
3. **Auction type**: First-price or second-price auction?
4. **Budget**: Real-time budget enforcement or eventual?

5. **Targeting**: What targeting dimensions? (geo, demographics, interests)
6. **ML**: Use ML for bid optimization?
7. **Fraud**: How to detect invalid traffic?

High-Level Design





API Design

Bid Request/Response APIs (Real-Time Bidding)

Method	Endpoint	Description
`POST`	`/api/v1/bid`	Submit bid request (from ad exchange)
`POST`	`/api/v1/win`	Win notification
`GET`	`/api/v1/ad/{adId}/render`	Render ad creative

Bid Request (OpenRTB-compatible)

```
POST /api/v1/bid
{
  "id": "req_123",
  "imp": [{
    "id": "imp_1",
    "banner": { "w": 300, "h": 250 },
    "bidfloor": 0.50
  }],
  "site": { "domain": "example.com", "cat": ["IAB1"] },
  "user": { "id": "user_anon_456" },
  "device": { "ua": "...", "geo": { "country": "US" } }
}

Response: 200 OK (< 100ms)
{
  "seatbid": [{
    "bid": [{
      "id": "bid_789",
      "impid": "imp_1",
      "price": 1.25,
      "adm": "<creative_markup>"
    }]
  }]
}
```

Campaign Management APIs

Method	Endpoint	Description
`POST`	`/api/v1/campaigns`	Create an ad campaign
`PUT`	`/api/v1/campaigns/{campaignId}`	Update campaign settings
`PUT`	`/api/v1/campaigns/{campaignId}/budget`	Update budget/pacing
`GET`	`/api/v1/campaigns/{campaignId}/stats`	Get campaign performance

Bid Request/Response Format

```
// BID REQUEST (from Ad Exchange)
{
  "id": "req_abc123",
  "imp": [{
    "id": "imp_1",
    "banner": {"w": 300, "h": 250},
    "bidfloor": 0.50
  }],
  "site": {
    "domain": "news.example.com",
    "page": "https://news.example.com/article/123",
    "cat": ["IAB12"]  // News category
  },
  "device": {
    "ua": "Mozilla/5.0...",
    "ip": "203.0.113.50",
    "geo": {"country": "USA", "region": "CA", "city": "SF"}
  },
  "user": {
    "id": "user_xyz789",
    "buyeruid": "dsp_user_456"  // DSP's user ID
  }
}

// BID RESPONSE
{
  "id": "req_abc123",
  "seatbid": [{
    "bid": [{
      "id": "bid_001",
      "impid": "imp_1",
      "price": 2.50,
      "adm": "<script>...</script>",  // Ad creative
      "nurl": "https://dsp.com/win?price=${AUCTION_PRICE}",
      "adomain": ["advertiser.com"]
    }]
  }]
}
```

Budget & Pacing

```
┌─────────────────────────────────────────────────────────────────────────────┐
│                          BUDGET MANAGEMENT                                  │
└─────────────────────────────────────────────────────────────────────────────┘

REAL-TIME BUDGET TRACKING:
┌─────────────────────────────────────────────────────────────────────────────┐
│                                                                             │
│   Redis Atomic Operations:                                                  │
│                                                                             │
│   # Check and reserve budget atomically
```

```
│   DECRBY campaign:123:budget 250  # Deduct bid amount (cents)
│   → Returns new balance
│   → If negative, bid was invalid (rollback)
│
│   # On auction win
│   DECRBY campaign:123:budget -250  # Refund reserved amount
│   DECRBY campaign:123:budget 150   # Charge actual price (2nd price)
│
└──────────────
```

```
    │  Solution: Multi-task learning, transfer learning
    └──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘

3. BID LANDSCAPE
    ┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
    │  Predict: P(win | bid_price)
    │  Use: Optimize bid to maximize ROI
    │
    │  optimal_bid = argmax(P(win|bid) × value - bid × P(win|bid))
    └──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

Key Components

Component	Purpose	Technology
Bid Service	Process bid requests	Go/Rust (low latency)
Filter Engine	Fast campaign matching	Bloom filters + rules
ML Scoring	Predict CTR/CVR	TensorFlow Serving
Budget Tracker	Real-time spend	Redis Cluster
Feature Store	Fast feature lookup	Redis / Aerospike
Campaign DB	Campaign configs	PostgreSQL + cache
Event Pipeline	Track impressions/clicks	Kafka → ClickHouse
Reporting	Analytics dashboards	ClickHouse + Grafana

Auction Types

```
SECOND-PRICE AUCTION (Traditional):
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│  Bids: A=$3.00, B=$2.50, C=$2.00
│  Winner: A
│  Pays: $2.51 (second price + $0.01)
│
│  Pros: Truthful bidding (bid your true value)
│  Cons: Revenue not optimal for exchange
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘

FIRST-PRICE AUCTION (Modern trend):
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│  Bids: A=$3.00, B=$2.50, C=$2.00
│  Winner: A
│  Pays: $3.00 (what they bid)
│
│  Pros: Simpler, more revenue
```

```
  Cons: Requires "bid shading" (bid below true value)
  
  BID SHADING MODEL:
    shaded_bid = true_value × (1 - shade_factor)
    shade_factor = f(market_competitiveness, historical_win_rate)
```

Scalability Considerations

- **Stateless bidders**: Any server can handle any request
- **Geo-distributed**: Deploy close to exchanges (reduce latency)
- **Feature caching**: Pre-compute user features, cache in Redis
- **Model serving**: Use TensorFlow Serving with batching
- **Circuit breakers**: Skip slow campaigns, maintain SLA
- **Sampling**: For very high QPS, sample requests (bid on 10%)

Trade-offs

Decision	Option A	Option B	Recommendation
Auction	Second-price	First-price	**First-price** - industry trend
Budget	Eventual	Real-time (Redis)	**Real-time** - avoid overspend
ML inference	Per-request	Batched	**Batched** - 10x throughput
Language	Java	Go/Rust	**Go** - fast, simple, GC-friendly
Feature store	Redis	Aerospike	**Redis** - simpler, good enough

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Kafka	Event pipeline for impressions, clicks, and conversions to billing/reporting	Tech Guide
Redis	Real-time budget tracking with atomic counters, feature store for ML	Tech Guide
Flink/Spark	Model training and feature engineering for CTR/CVR prediction	Tech Guide

How to Explain It Well

> **Opening**: "RTB systems must respond in <100ms. I'll design a bidding engine that filters 90% of campaigns on rules, then uses ML for the rest."

> **Key talking points**:
>
> - "Fast filtering first: budget, geo, frequency cap - eliminates 90% of campaigns in <5ms"
> - "ML scoring for remaining campaigns: pCTR × pCVR × value = expected value"
> - "Redis atomic counters for real-time budget - can't afford eventual consistency"
> - "Pacing algorithm spreads budget evenly throughout the day"
> - "Second-price auction encourages truthful bidding; first-price needs bid shading"
> - "Feature store (Redis) for <1ms feature lookup - can't afford DB calls"

Common Follow-up Questions

1. **How do you handle latency spikes?**
    - Set hard timeout (50ms) - return no-bid if exceeded
    - Circuit breaker for slow models/services
    - Pre-compute as much as possible
2. **How do you prevent overspending?**
    - Redis atomic decrements with rollback on loss
    - Conservative budget reservation (reserve full bid, refund on 2nd price)
    - Async reconciliation every hour
3. **How do you detect ad fraud?**
    - IP/device fingerprint analysis
    - Click pattern anomalies (too fast, same user)
    - Third-party verification (IAS, DoubleVerify)
    - Conversion verification with advertiser